The Embedded-Chain RGA

A tombstone-free replicated list whose sort keys are the birth chains, entropy-coded: design, proofs, validation, and related work

Sal / FP Launchpad (KC + Claude)
working names in the artifact: `embed-tree` (unary), `embed-code` (delta-coded)

2026-07-14; revised 2026-07-15 (the datatype separated from its encoding)

Abstract

The embedded-chain RGA (replicated growable array) is a replicated list that keeps no tombstones, no ledger, and no event log: its entire state is the live nodes. It is a sequence MRDT, a *mergeable replicated data type*: state-based, reconciled by a three-way merge against the branches' lowest common ancestor version. A deleted element survives only as $\Theta(\log \delta)$ bits inside its descendants' sort keys, δ being the Lamport-timestamp gap between the element and its own insertion anchor (1 under sequential editing); that is the entropy of the race that created it, paid once, in space. Concurrent inserts provably cannot tie, so the merge contains no repair and never decides an order; the state is a canonical function of the event set, and convergence is literal equality. Observationally the datatype *is* the published RGA with the tombstone set compressed into coordinate entropy: a kernel-checked read-equivalence theorem, with machine lockstep at every step as its tested form. All of this follows from one idea: an element's sort key is its *birth chain*, the timestamps from the root of the birth tree down to the element, dead ancestors included. The chain is written at birth, never edited, and carried in an order-preserving self-delimiting code. Separating the datatype from its encoding makes the metatheory nearly definitional: correctness is three one-line facts about chains, plus a single faithfulness theorem about the code. The design is machine-validated (the L1–L25 anomaly battery, except L19, RGA's own; 420+ randomized version-DAG executions; 120/120 lockstep), and the central theorem, RA-linearizability at every honestly reachable configuration, is a kernel-checked Lean theorem (§7).

1 The datatype: sort keys are birth chains

Timestamps $t \in \mathbb{N}^+$ are Lamport stamps, globally unique, and double as element identities. Causality gives: the anchor of every insert carries a smaller timestamp than the insert itself.

The birth tree. Every insert names an anchor. The event set therefore induces a tree: the root is the front sentinel 0; the *birth parent* of each element is its anchor. The tree is append-only and never edited: an element's place in it is fixed at birth.

Birth chains. The *birth chain* of u is the sequence of timestamps from the root down to u (sentinel elided): $\mathrm{chain}(u) = \mathrm{chain}(a) \cdot u$ when u was inserted after anchor a, and $\mathrm{chain}(u) = \langle u \rangle$ at the front. Dead ancestors are *not* elided: their timestamps remain in their descendants' chains. The chain is the sort key, and everything in this section is stated in terms of it; nothing here depends on how chains are stored (§2).

Type. A state is a finite map over *live nodes only*:

$$\sigma \;:\; u \;\mapsto\; (\,\mathrm{chain}(u),\ \mathrm{char}_u\,).$$

init. $\sigma_0 = \varnothing$.

do (the operations). Two operations, with $\mathsf{rc} = \mathsf{Either}$ everywhere (§3):

- $\mathsf{ins}(x \leftarrow a,\ c,\ \pi)$: insert element x (its timestamp) with character c after anchor a ($a = 0$ for the front). The payload $\pi = \mathrm{chain}(a)$ is the anchor's birth chain, carried *for the proof alone* (§3). Application on a state where a is live (every reachable application, by honesty):

  $$\sigma[x] := \big(\mathrm{chain}_\sigma(a) \cdot x,\ c\big), \qquad \pi \text{ unread.}$$

  No head-tracking, no reading of siblings.

- $\mathsf{del}(d)$: remove d's record. Nothing else changes: survivors' chains are untouched; in particular, d's descendants still carry d's timestamp in their sort keys. If d is absent, no-op (concurrent double delete).

read. Sort survivors by *chain-lex*: compare chains timestamp-by-timestamp with *larger first*; a proper prefix precedes its extensions (a live ancestor displays immediately before its descendants). Among same-parent siblings this is exactly RGA's newest-first rule; equivalently, the canonical order is the RGA traversal of the birth tree restricted to survivors.

merge. merge(L, A, B), where L is the LCA (the branches' lowest common ancestor version, the merge context the MRDT model stores and supplies). The merge is *survival* (OR-set): live iff in all three, or born in exactly one branch. There is no second step at this layer: survivors' chains come with their insert events, so every input that holds a survivor agrees on its chain. The merge never compares timestamps and never decides an order.

Inv / applicable / honesty. The honesty contract is the standard RGA generation discipline: an insert is *generated* at a replica where its anchor is live (you insert after a character you can see), and $\mathsf{ins}/\mathsf{del}$ are *applied* on states where they are applicable (anchor live; delete of a live or already-dead node). On every reachable state, application never consults π. Canonicity (Theorem 1) doubles as Inv; at the representation layer, Theorem 2(iii) is its stored form.

$\approx$. Equality. The state is a canonical function of the event set (Theorem 1), so no quotient is needed: convergence is literal state equality.

The metatheory, which is nearly definitional. At this layer the design's theorems all but evaporate; we record them because they are the specification the representation must meet (§2).

Theorem 1 (chain-level metatheory)**.** *In every reachable state:*

(i) Canonicity: $\sigma = \{\, u \mapsto (\mathrm{chain}(u),\ \mathrm{char}_u) \,\}$ *over survivors* u*; the state is a function of the event set.*

(ii) Birth constancy: $\mathrm{chain}(u)$ *is fixed at* u*'s insert and never edited by any later transition.*

(iii) Totality (no ties): chain-lex is a total order on every state's survivors.

Proof. (i) Survivorship is OR-set, hence a function of the event set, and a survivor's chain is determined by its insert event alone: by induction, ins writes $\text{chain}_\sigma(a) \cdot x = \text{chain}(a) \cdot x$ (the anchor is live at application, by honesty, and its stored chain is canonical), while del and the merge only delete or select records, never rewrite them. (ii) is (i) read off per node: no transition writes to an existing record. (iii) Distinct elements' chains are distinct (they end in distinct timestamps); if neither is a prefix of the other, the first difference compares two distinct timestamps in $\mathbb{N}$, and uniqueness of Lamport stamps *is* the no-tie property; prefix pairs are ancestor/descendant, ordered by the prefix rule. □

The one-line separation, and the conservation law. The natural first attempt at tombstone-freedom is the *rehoming* RGA: keep flat (element, anchor) records, and let a delete splice its node out, re-parenting the orphans to the nearest live ancestor (the *climb*). We built and verified that design before this one (it is RA-linearizable, in the same Lean framework, §7), and it fails the L1 delete-reorder litmus (the $[b, a, c] \to [c, b]$ anomaly, Figure 1): the climb re-sorts a rehomed node by its own timestamp, so *its sort key is the birth chain truncated at each rehoming.* This design keeps the chain whole. That is the entire difference between the two datatypes, and it is the crisp form of the conservation law that our design sweep kept re-imposing: *the sort key must retain dead ancestors' timestamps.* Every design that forgets them has a named machine-checked countermodel (§6). What is negotiable is only how many bits the retained timestamps cost; that is the subject of §2, and the answer is: the entropy of the races, and nothing else.

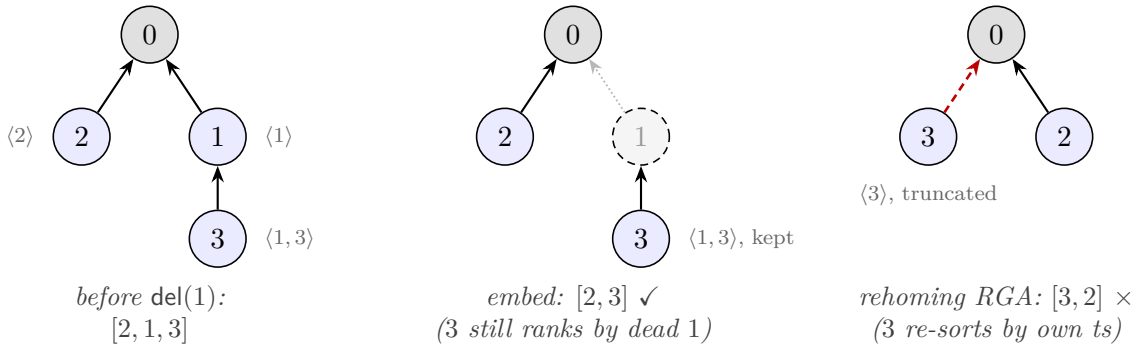

Figure 1: The one-line separation on the L1 delete-reorder litmus: insert 1 and 2 at the front, 3 after 1, delete 1. The rehoming RGA's climb (red) re-parents 3 and re-sorts it by its *own* timestamp: the truncated key $\langle 3 \rangle$ beats $\langle 2 \rangle$, and 3 jumps over 2. The embedded chain $\langle 1, 3 \rangle$ keeps ranking 3 by its dead parent's timestamp: $[2, 3]$, the delete-order-preserving read. Arrows point from a node to its anchor; dashed grey = dead.

Worked example (Figure 2). 6 and 10 concurrently insert at the front: chains $\langle 6 \rangle$ and $\langle 10 \rangle$; display $[10, 6]$, no arbitration, at every replica and merge order. Type 22, 16 under 6: chains $\langle 6, 22 \rangle$, $\langle 6, 16 \rangle$. Delete 6: the records of 22 and 16 are untouched and still carry 6's timestamp. A third party concurrently inserts 8 at the front: $\langle 8 \rangle$ sorts below $\langle 10 \rangle$ and above $\langle 6, 22 \rangle$, so the display is $[10, 8, 22, 16]$ whether 8 arrives before or after 6's death (machine-checked as the *credential countermodel*, the shape that kills every timestamp-oblivious variant, §4).

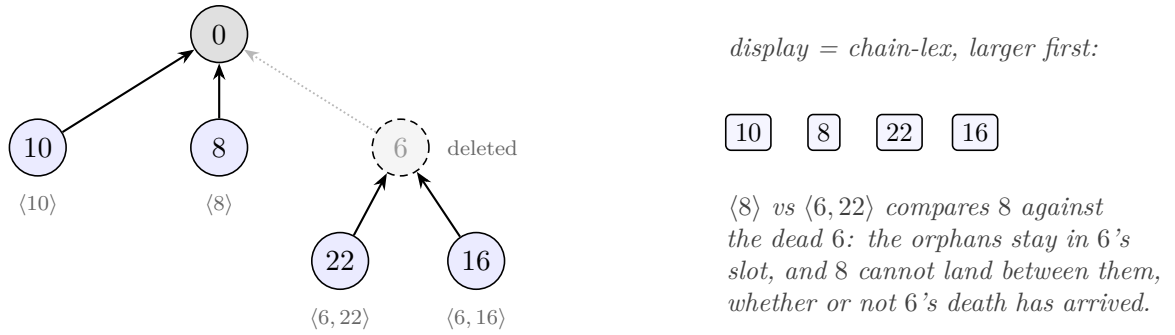

Figure 2: The credential countermodel as a birth tree. 6 and 10 race at the front; 22, 16 are typed under 6; 6 dies; 8 arrives at the front concurrently. Survivors sort by whole birth chains, so the read is $[10, 8, 22, 16]$ at every replica and merge order. Every timestamp-oblivious variant flips this shape (§6).

2 The representation: chains as entropy-coded coordinates

Everything above is the datatype; everything below is bits. A naive carrier would store a full timestamp at every chain level (each $\sim \log$(document age) bits, even for purely sequential typing) and compare whole chains at every read. This section stores the chains compactly without disturbing their order.

Vocabulary. A *codeword* is the bit string $C(\delta)$ encoding one chain level: one element's timestamp, stored as its differential δ to its birth anchor (step 1 below). A *coordinate* is the bit string encoding a whole chain: the concatenation, root to element, of one block per level (the level's codeword plus three framing bits fixed by the mint construction below), with nothing else marking the boundaries. We write $|s|$ for the length in bits of a bit string s. Read as a binary fraction, a coordinate names a dyadic interval in $(0, 1)$; codeword concatenation becomes interval nesting, and the interval is the $\alpha(u)$ of Theorem 2. Bit string and interval are two views of one object; we use whichever is clearer.

What the coding is for. Two jobs, only one of which is about correctness. *Job 1: cost.* The datatype layer already forced sort keys to retain dead ancestors' timestamps (the conservation law, §1); the only remaining question is how many bits that retention costs. Without entropy coding, "tombstone-free" would be bookkeeping: a full timestamp per level grows with document age however calm the history, and the unary control below costs 61 KB where the coded design costs 0.5 KB. The coding is what makes the headline claim true: the dead survive as the entropy of the races that created them, and not more. *Job 2: faithfulness.* The chains must be stored so that plain lexicographic bit comparison reproduces chain-lex exactly, in every reachable state, forever. This is the single obligation the representation owes §1 (Theorem 2). Correctness lives upstairs; this layer is an implementation that must not lie. Job 2 needs no entropy optimality at all: it needs exactly two structural properties of the per-level code, stated next.

The contract. The stored form must support four operations: *mint* (insert: produce the new codeword from the two timestamps alone, coordination-free, reading no siblings); *compare* (read: plain lexicographic comparison, with no tie rule); *fold* (delete: splice a dead node out by concatenating its block into each child's stored string, exactly); and *refold* (merge: recompute survivors' coordinates by the same concatenation). Each guarantee is bought by a specific property of C:

- **Monotone** ($\delta < \delta' \Rightarrow C(\delta) <_{\text{lex}} C(\delta')$) buys correct *compare*. A chain-lex first difference always compares two same-anchor entries, where timestamp order and δ order coincide (step 1); bitwise comparison of coordinates therefore equals timestamp comparison at the first differing level precisely when C is monotone.
- **Prefix-free** (no codeword is a prefix of another) buys three things at once. *Alignment*: in a comparison of two coordinates, the first differing bit falls inside the first differing level's codewords, so levels cannot be confused across boundaries and no delimiters are needed; this is also what keeps *fold* and *refold* exact splices. *Geometry*: by the Kraft correspondence, prefix-free codewords occupy pairwise-disjoint dyadic cells, which is Theorem 2(i) and the reason a concurrent front-insert can never land inside a dead sibling's span (Figure 4). *No ties*: cell disjointness plus Lamport uniqueness means sibling coordinates never tie, so the merge never decides an order and the read needs no tiebreak.
- **Universal, with a cheap floor** buys Job 1: C is defined for every $\delta \geq 1$, $|C(1)| = O(1)$ so sequential typing costs a constant per level, and $|C(\delta)| = \Theta(\log \delta)$ so a race pays for its own magnitude and nothing else.

Monotone plus prefix-free is all of Job 2: any such code satisfies Theorem 2, which is why the Lean capstone is parametric in the code (§7) and why the unary control proves the same theorems at absurd cost. Entropy coding enters only in *which* monotone prefix-free code is chosen; the choice sets the constant in the bits-per-level bill and nothing in the correctness story. One requirement cuts across all four operations: the arithmetic is exact, never rounded (see the binary64 refutation below).

Step 1: differential coding. By causality $\delta = x - a \geq 1$. A chain-lex first difference always compares two entries with the *same* anchor (the shared prefix), and for a fixed anchor, ordering by x and by δ coincide, so each chain level may store the differential instead of the timestamp. This is where *pay for concurrency* enters: sequential typing has $\delta = 1$ at every level regardless of how long the document has lived, while a genuine race with timestamp gap g costs $\log_2 g$ bits. The differential is an encoding choice, not semantics: the mathematical object remains the timestamp.

Step 2: entropy coding. For $\delta \geq 1$ let L be the bit-length of δ and

$$C(\delta) \;=\; \underbrace{1\cdots1}_{L-1}\;0\;\big(\delta \text{ with its leading bit removed}\big) \qquad (|C(\delta)| = 2L-1),$$

an order-preserving prefix-free code: exactly the contract. The length accounting: prefix-freedom makes each codeword self-delimiting, and a self-delimiting code over unbounded δ must spend bits twice, once on the value and once announcing its own length. C pays each bill at $\sim\log_2 \delta$. The header $1^{L-1}0$ is a unary announcement "$L{-}1$ payload bits follow" (L bits); the payload is δ's trailing $L{-}1$ bits, since the leading bit of an L-bit number is always 1: once the header has said L it carries no information and is dropped. Hence

$$|C(\delta)| \;=\; \underbrace{(L{-}1)+1}_{\text{header}} + \underbrace{(L{-}1)}_{\text{payload}} \;=\; 2L-1 \;=\; 2\lfloor\log_2 \delta\rfloor + 1,$$

e.g. $C(1) = 0$, $C(2) = 100$, $C(3) = 101$, $C(5) = 11001$ (the values kernel-checked in the artifact). This is Elias γ with its header flipped: γ announces the length in *zeros*, which is prefix-free but order-reversing across length classes: at the first divergence the longer, hence larger, number shows

a header 0 against the shorter's leading payload bit 1. Writing the header in ones makes that same position decide in favor of the longer code, so lexicographic order on codewords is numeric order on δ. The γ-flip is the building block, not the last word: only the *value* bill is forced at $\log_2 \delta$, and the *length* bill can itself be entropy-coded. Applying C to the length field (then the payload) gives the order-preserving flip of **Elias** δ, prefix-free and monotone by the same across-length-class argument, at $\log_2 \delta + O(\log \log \delta)$ per level; *this is the canonical mint*, the best proved code so far. *Any* order-preserving prefix-free code works here, and the artifact exists in three proved encodings of one datatype (`embed-tree` unary, `embed-code` the γ-flip, `eliasDeltaCode` the δ-flip); the capstone theorem is inherited at each verbatim, with zero new proof content (§7): the code-parametricity claim, machine-validated. Two honest footnotes: per level the δ-flip wins only from $\delta \geq 32$ and loses at $\delta \in \{2, 3\} \cup [8, 15]$ (kernel-checked length comparisons), and measured editing tails are thin, so on real traces the two flips are a wash; the dominance shows in race-heavy shapes (the entropy-law table below) and in the asymptotics.

The mint. Writing $b = 1 \cdot C(\delta)$ (a leading 1 keeps coordinates positive) and $w = 2^{-|b|-2}$, the mint is the second quarter of b's dyadic cell:

$$M(\delta) \;=\; \big(\, 0.b + w, \quad 0.b + 2w \,\big) \;\subset\; \big[\, 0.b, \;\; 0.b + 4w \,\big).$$

The quarter placement leaves a gap below and headroom above inside the cell, and the open space $(0.b + 2w,\ 0.b + 4w)$ is never minted: it exists so that the interval has room to *contain* the node's own subtree without touching the next cell.

Figure 3: Mints $M(\delta)$ for $\delta = 1..4$ inside the parent's unit interval (cell boundaries dotted). Larger deltas sit strictly higher; distinct deltas are disjoint with gaps; each mint is a quarter of its cell, leaving room for the node's own subtree. Drawn for the γ-flip C; the construction reads verbatim over any order-preserving prefix-free code, in particular the canonical Elias-δ mint (step 2).

The stored state. The carrier stores each live node parent-relatively:

$$\sigma \;:\; t \;\mapsto\; (\,\mathrm{par} \in \mathbb{N}, \quad (lo, hi) \subset (0, 1), \quad \mathrm{char}\,),$$

where (lo, hi) is a dyadic-rational interval *relative to the parent's interval*, and par $= 0$ denotes the root sentinel (which owns $(0, 1)$ and is never stored). A node's *absolute* interval is the affine composition of the intervals along its parent chain; write

$$\alpha(u) \;=\; \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}\big(M(\delta_u)\big)$$

for the composition of the mints along u's birth chain $c_1 \cdots c_k\, u$ (where φ_v maps $(0, 1)$ onto v's mint interval, order-preservingly, and δ_v is v's delta to its birth parent): $\alpha(u)$ is the *encoding of* chain(u). Semantically the coordinates are rationals; the natural carrier is a bit string per node (below). Figure 4 draws the worked example of §1 in coordinates.

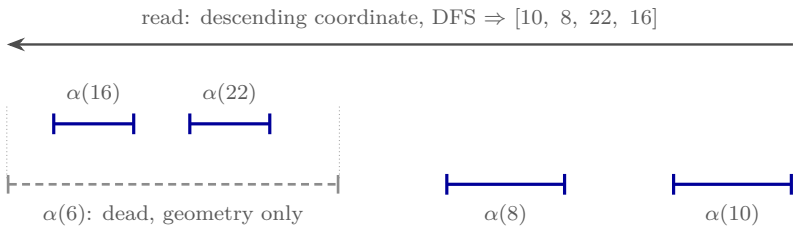

Figure 4: Figure 2 downstairs (widths schematic; true mints are the dyadic cells of Figure 3). Deleting 6 removes its record, but $\alpha(16)$ and $\alpha(22)$ are compositions *through* $M(6)$: the dead node survives as the dashed container, the tombstone compressed into geometry. 8's interval can never land inside $\alpha(6)$'s span: distinct deltas occupy disjoint cells (Theorem 2(i)), which is the credential verdict, now a fact about intervals.

Maintenance. How the stored form tracks the datatype:

- $\mathsf{ins}(x \leftarrow a)$ mints $M(x-a)$ relative to a: the new record depends only on the two timestamps.
- $\mathsf{del}(d)$ *isometrically folds* d into its children: each child c is re-parented to d's parent with d's interval composed in, $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$, an exact affine substitution. Note what this is: upstairs, deletion changes nobody's sort key (§1); the fold is the parent-relative storage scheme *re-normalizing* while every absolute interval stays fixed (Figure 5). Representation maintenance, not datatype semantics.
- The merge *refolds*: for each survivor, compute its absolute interval by folding its record to the root inside the input that holds it (a record's stored parent is always live in its own input, so the chain stays in one input; by Theorem 2(iii) the choice of input cannot matter, asserted at runtime and never fired); re-parent to the nearest *surviving* ancestor; re-relativize.
- The read is depth-first from the root, children in descending order of *lo*. No tie rule is needed (Corollary 1).

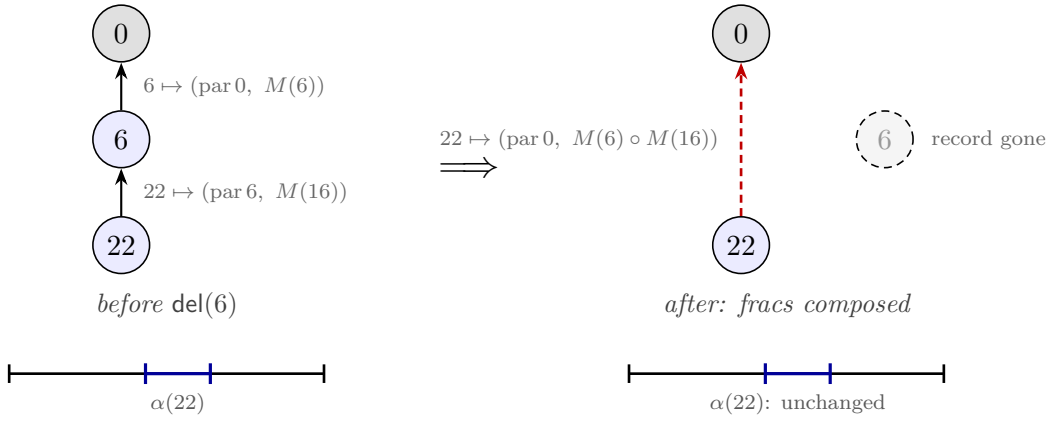

Figure 5: The isometric fold. $\mathsf{del}(6)$ deletes 6's record and substitutes its stored interval into each child's: $22.\text{frac} := M(6) \circ M(16)$, an exact affine composition (red = the re-parenting, a storage move only). The absolute interval $\alpha(22) = M(6) \circ M(16)$, and with it the display, is bit-for-bit unchanged: upstairs, 22's sort key still reads $\langle 6, 22 \rangle$.

The one obligation. The four theorems of the earlier drafts collapse into a single statement about the encoding.

Theorem 2 (encoding faithfulness)**.** *(i) Order-embedding at one level: for $\delta \neq \delta'$, $M(\delta)$ and $M(\delta')$ are disjoint with a gap; $\delta < \delta' \Rightarrow M(\delta)$ lies entirely below $M(\delta')$.*

(ii) Homomorphism: α realizes chain concatenation as affine composition and embeds chain-lex in the interval order: for distinct u, v, if $\mathrm{chain}(u)$ *is a proper prefix of* $\mathrm{chain}(v)$ *then $\alpha(v) \subset \alpha(u)$ (and, if both are live, the two are never read-time siblings); otherwise $\alpha(u) \cap \alpha(v) = \varnothing$, with the chain-lex-greater element's interval strictly higher.*

(iii) Exact maintenance: in every reachable state, every live node's stored records compose to exactly $\alpha(u)$.

Proof. (i) C is prefix-free: codes of lengths $2L-1 < 2M-1$ differ at position L (the shorter's header terminator 0 against the longer's header 1); equal-length codes differ in the payload. Prefix-free codes have pairwise-disjoint dyadic cells. C is monotone as a binary fraction: within a length class the payload is δ's low bits in order; across classes the position-L bit decides in favor of the longer code. Disjoint cells ordered by their codes, and mints interior quarters of their cells, give both claims. (ii) If neither chain is a prefix of the other, they first differ on distinct deltas relative to the *same* birth ancestor (§2, step 1); (i) separates them there, and affine images of disjoint intervals are disjoint and order-preserved (each φ is increasing). If $\mathrm{chain}(u)$ prefixes $\mathrm{chain}(v)$, then $\alpha(v) \subseteq \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}(\varphi_u((0,1))) = \alpha(u)$ by construction; and if both are live, v's nearest live ancestor is u or deeper, so they never share a current parent, and sibling groups are pairwise disjoint. (iii) By induction over transitions. ins establishes it by definition. del's fold replaces $(\mathrm{par}, \mathrm{frac})$ pairs by their exact composition, leaving α unchanged. The merge recomputes precisely $\alpha(u)$ (folding a record chain to the root) and re-relativizes against another α; ties cannot exist by (i) and (ii), so no step of any transition ever rewrites an interval by anything but exact composition. □

Corollary 1 (display ≡ chain-lex; no tie rule)**.** *The read (DFS by descending lo) enumerates survivors in exactly chain-lex order: per level, larger delta first, so among same-parent siblings, exactly newest first; survivors rank by their own timestamps, folded heirs by their dead founder's. Sibling lo-values never tie; the read's id tiebreak is dead code. Machine checks: pairwise sibling disjointness asserted at every read of* 320 *randomized executions, all clean; over* 19,531 *multi-member sibling groups, own-timestamp order violated* 3,481 *times (exactly the post-fold heir groups; forcing own-ts there is the refuted dissolution behavior),* ***chain-lex violated*** 0 ***times****.*

The entropy law. A coordinate is the concatenation of the codes along the birth chain: $\Theta(\sum_i \log \delta_i)$ bits, the entropy of the chain. Sequential typing ($\delta = 1$ at every level) costs ∼4 bits per level under every code here; a race with timestamp gap g costs $\log_2 g + O(\log \log g)$ bits under the canonical Elias-δ mint ($2 \log_2 g + 1$ under the γ-flip). *The state pays for the magnitude of actual concurrency, and for nothing else.* Measured (1000 elements; the numbers are *order metadata alone*, survivors' coordinate bits read off the dyadic denominators; characters are excluded, since data costs are identical across all designs):

scenario	quarter carve (refuted)	unary mint	γ-flip mint	**Elias-δ mint**
chain then delete-all (δ=1)	2^{2001}	2^{502501} (∼61 KB)	2^{4001} (∼0.5 KB)	$\mathbf{2^{4001}}$ (∼0.5 KB)
1000 root siblings (δ=t)	— (ties)	max 2^{1003}, 125 KB	max 2^{23}, 5 KB	**max $\mathbf{2^{20}}$, 4 KB**

The carrier. The natural representation is a **parent-relative bit string per node**: the fold is concatenation, comparison is lexicographic ($O(1)$-ish with structure sharing); the rationals are the semantics of the strings, kept for the proofs. IEEE 754 binary64 is machine-refuted as the carrier, a faithfulness failure rather than a design failure: mints degenerate at $t = 52$, depths underflow at 44, and rounding voids Theorem 2(iii) (6/120 PBT executions die). The obstruction is a pigeonhole (chains carry $\Theta(\sum \log \delta)$ bits; a fixed-width word holds 64), but binary64 is sound as a *comparison cache* with exact fallback on float equality.

Timestamps in practice. The unique stamps in $\mathbb{N}^+$ are Lamport's 1978 construction, flattened: each replica ticks a counter per local operation, advances it to the max on merge, and breaks ties by replica id, packed as $t \cdot 2^k + r$ with k id bits. Uniqueness and causality ($t_a < t_x$ gives $\text{packed}_a < \text{packed}_x$ regardless of ids, so $\delta \geq 1$) both survive the packing; the proof layer only *assumes* uniqueness (the Lamport-structural part of honesty), and this construction is the witness that the assumption is realizable. Two entropy caveats. *(i)* Packing inflates every delta by 2^k, costing $\sim 2k$ extra bits per chain level, which turns the 0.5 KB sequential run above into $\sim$3 KB at $k = 10$. Cheaper: extend the mint's code to the pair, $C(\Delta t)$ followed by a k-bit id. The pair code is still order-preserving prefix-free (lex on $(\Delta t, r)$), and costs k bits only where the level is minted. A further option, unvalidated against the battery and recorded as a design note, is to order sibling ids *relative to the anchor's* (any deterministic per-anchor id order is a legitimate tiebreak; convergence needs agreement, not a uniform order), making "same author as my anchor", i.e. sequential typing, cost $\sim$1 bit, so only genuine cross-author races pay $\log R$. *(ii)* The clock must be *dense logical time, never physical*: wall-clock or HLC stamps make δ measure elapsed time rather than elapsed events: a keystroke every 100 ms over μs-resolution stamps mints $\delta \approx 10^5$, $\sim$34 bits per level for a purely sequential edit. The entropy law holds precisely because the counter ticks once per event, making δ an event-count gap; if wall-clock order is ever wanted, it belongs in a display-layer tiebreak, never in the minted coordinate.

What remains open. Dead chain entries concatenated into survivors' strings are the conservation law's irreducible content (§1); reclaiming them under *causal stability* (renumber once every replica has seen the delete, the only condition under which rewriting geometry is safe, since no verdict involving the dead can be contested again) is the deepest of the open cost items, and it is orthogonal to correctness. The wider compression headroom is catalogued with its research questions in the companion assessment `whiteboard/order-coding-compression-notes.md`: context-conditioned per-anchor codes (licensed by comparison locality: a chain-lex first difference never crosses anchors), coalescing δ=1 runs, Steiner-sharing dead prefixes across co-heirs, stability-gated rank re-coding, and the Elias-δ constant (step 2 above).

3 The insert prefix: for the proof, and the proof alone

The insert payload π exists for verification, not execution. This section says why it must exist, and why an implementation may drop it from the wire.

Why rc must be Either. The verification framework (§7) resolves a concurrent pair of non-commuting operations by consulting a declared conflict-resolution order rc; any *oriented* entry for an insert/delete pair, however, incurs a base commutation obligation that is machine-refuted for this design space: ordering $\mathsf{ins}(x \leftarrow a)$ against $\mathsf{del}(a)$ (in either direction) is undischargeable as a

conflict entry, because rehoming is non-transitive across chains of deletions. So rc = Either: the proof must show the two *commute*, in both orders, on all applicable states.

The commutation, and where the prefix earns its keep. Order 1, ins($x \leftarrow a$) then del(a): x is born with chain $\pi \cdot x$, and the delete touches no chain (downstairs: the fold re-homes x's record with composed coordinates; by exactness $\alpha(x) = \alpha(a) \circ M(x - a)$). Order 2, del($a$) then ins($x \leftarrow a$): a is gone; application must still be *total* and land x on the same sort key. This is what the carried prefix π = chain(a) (equivalently, the anchor's coordinate) provides: apply sets chain(x) = $\pi \cdot x$ (downstairs: $\alpha(x) = \pi \circ M(x - a)$, attached to the nearest live ancestor). The two orders produce *equal* states: commutation to equality, not merely up to $\approx$, courtesy of canonicity (Theorem 1). Figure 6 draws the square on the smallest instance.

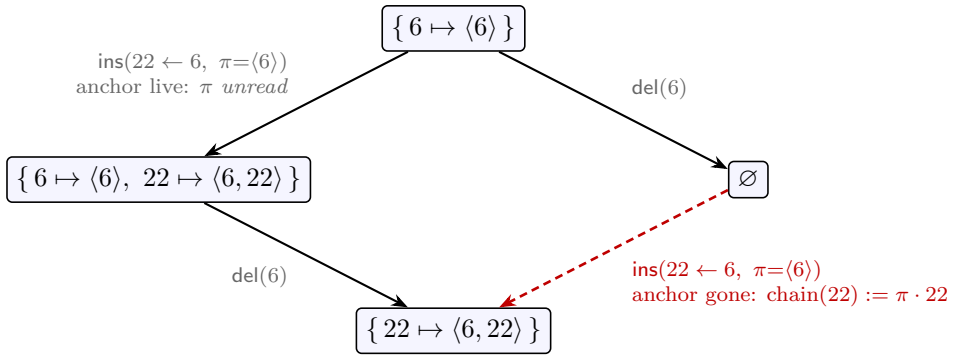

Figure 6: rc = Either discharged: ins($22 \leftarrow 6$) and del(6) commute *to equality*. Down the left, the anchor is live and application never consults π (the reachable, honest order). Down the right, 6 is already gone: the state cannot supply the anchor's chain, so the carried π = chain(6) does, landing 22 on the same sort key. The right-hand state is unreachable under honesty; the verification condition quantifies over it anyway, and π exists exactly to make that corner total.

Ghost, in the strict sense. On every reachable state the anchor is live at application (honesty), and the application reads only the state and the two timestamps: π is never consulted. The Lean artifact carries this as `ins_prefix_ghost` (`Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean`): on accurate ops the always-π transition and the state-reading transition are literally the same function: the two conventions of this section coincide on every honest state. An implementation may therefore drop π from the wire entirely; the *proof* may not, because the verification conditions quantify over reorderings that visit unreachable states, and there application without π is partial. The prefix costs $O(\sum \log \delta)$ bits, the same entropy as the coordinate itself, and exists at the proof layer alone.

4 Validation

All numbers reproduce from `whiteboard/litmus/` in the accompanying Sal repository (`embed_tree.py`; harnesses `litmus.py`, `pbt.py`). Notation: RGA$_\dagger$ is the published tombstoned RGA (Roh et al.; §6); the *rehoming* RGA is the verified flat tombstone-free design of §1. The L1–L25 battery is the litmus suite accumulated over this project's design sweep; each entry is the named countermodel that killed at least one of 16 prior candidate designs.

check	verdict
litmus battery L1–L25 (incl. every countermodel that killed 16 prior designs: ceiling escapes, tie inheritance, three-branch topology, frame mixing, fold-verdict inheritance)	clean, except one-sided L19 (backward-run interleaving, the published RGA's own anomaly)
credential countermodel (tie heirs vs mid-ts third party, death-before vs death-after topologies)	converges, no flips; $= \mathrm{RGA}_{\dagger}$'s reads
subordination-escape and retroactive-subordination CEs	survive
randomized version-DAG PBT (FLIP / CONV / LIVE / DUP)	120/120 and 300/300 clean
per-read sibling-disjointness assertion sweep	320/320 clean
chain-lex instrumentation (19,531 sibling groups)	0 violations
lockstep vs the published tombstoned RGA	**read-equal at every apply/merge/read, 120/120**
L1 delete-reorder vs the verified rehoming RGA	embed $[2, 3]$ ✓ vs rehoming $[3, 2]$ ×

The lockstep row is the identification: *the embedded-chain RGA is observationally the published tombstoned RGA with the tombstone set compressed into coordinate entropy*, machine-checked on 120 executions and now a kernel-checked theorem (the compaction theorem, §7): on every honest event set, any enumeration of the tombstoned RGA's visible ids in its own visible order *equals* the embed read, element for element. The L1 row is the separation from the verified rehoming RGA, and it is exactly the one-line difference of §1: that design's climb re-sorts a rehomed node by its own timestamp (the $[b, a, c] \to [c, b]$ anomaly), i.e. its sort key is the birth chain *truncated at each rehoming*; this design keeps the chain whole.

5 Comparison with Eg-walker

Eg-walker (Gentle & Kleppmann, EuroSys 2025) is the closest system in spirit, and the two designs can be read as the *operational* and *denotational* realizations of one thesis, **pay for concurrency, not for history**:

	Eg-walker	embedded-chain RGA
model	op-based; replicas exchange events	MRDT: three-way merge with an LCA
durable state	**full event graph on disk**, deletes included, retained while concurrent ops may arrive; document state metadata-free	**live nodes only**; no event log anywhere; dead survive as $\Theta(\log \delta)$ bits inside live coordinates
merge mechanism	*replay*: walk the event graph, rebuild a transient CRDT structure (tombstones reappear transiently), discard at causally-linear points	*refold*: O(survivors) coordinate recomputation; no replay, no transient structure, no order decisions
sequential-editing cost	$\approx$ zero (plain edits; no metadata)	$\approx$ zero order metadata ($\sim$4 bits/level; plain append-like behavior)
concurrency cost	replay time proportional to the concurrent region, at every affected merge	coordinate bits proportional to log(ts gap), once, in space
ordering semantics	Yjs-variant (FugueMax-adjacent); maximal non-interleaving *conjectured*, not proved; two-sided (no L19 analogue)	$\equiv$ published RGA (machine lockstep); one-sided: L19 backward-run interleaving inherited, two-sided extension future work
correctness artifact	informal proof (Appendix C) + property testing	L1–L25 battery + DAG PBT + lockstep vs $\text{RGA}_\dagger$; **RA-linearizability kernel-checked in Lean** (§7); the design was chosen so the merge proves itself (no order decisions to verify)
op payload	integer origin positions (compact, human-meaningful)	two timestamps + ghost prefix (droppable on the wire; §3)

Three sentences of honest positioning. *(i)* Eg-walker achieves its clean document state by keeping the entire history and re-deriving order on demand; the embedded-chain RGA achieves it by making order derivation unnecessary: the sort keys are written at birth and never edited, and the coordinates are their encoding (§2), precomputed at mint time and exact forever. *(ii)* Eg-walker's ordering target is stronger where L19 is concerned (Fugue-style two-sidedness removes backward interleaving; this design inherits RGA's one-sided anomaly by construction, since it is read-equal to RGA). *(iii)* The verification gap runs the other way: Eg-walker's replay-transform-clear loop is algorithmically sophisticated and only informally proved; here the merge performs no arbitration at all, the metatheory is one-line at the chain layer plus a single faithfulness theorem at the encoding layer, and RA-linearizability is a kernel-checked theorem in a framework that had already verified a dozen-plus replicated datatypes end-to-end (§7). In the MRDT setting there is also a model fit: Eg-walker must reconstruct merge context by replay because the op-based model gives it none; the MRDT model hands the merge its LCA, which is exactly the context replay rebuilds.

6 Related work

The verified landscape sweep lives in `whiteboard/litmus/related-work.md` (per-claim verdicts, primary sources); this section positions the datatype.

Retention taxonomy. What nearby systems keep for the dead: tombstone records (RGA with cemetery under causal stability; WOOT; Yjs's per-deletion GC structs, kept forever; Fugue, "we cannot remove a deleted element's node entirely"; Peritext, "the positions must be remembered"); the full history (Eg-walker's event graph, Chronofold/causal trees where the log *is* the text); consensus-gated compaction (Treedoc's flatten requires Paxos-commit); or **dead identifiers inside live position keys** (Logoot/LSEQ, Weidner's position-strings/list-positions, the Fugue id space). The embedded-chain RGA is in the last class *by information content* (dead ancestors' timestamps persist inside survivors' sort keys, at live-reachable scope), but with three deltas to its classmates: the retention is *delta-coded at the entropy of the races* (a sequential history costs a constant per element where Logoot-family keys grow with allocation policy), the *birth tree kept whole* gives forward non-interleaving and delete-order preservation (position-key designs interleave, the PaPoC'19 anomaly, and Figma-style fractional indexing is unsound under concurrency by its own account), and the semantics is pinned to the best-studied sequence CRDT by a machine-checked lockstep rather than being a new point in anomaly space.

The impossibility backdrop. Attiya et al. (PODC 2016) prove $\Omega(D)$ metadata for push-based protocols meeting even the weak list spec: the published form of "ordering requires remembering the dead." This design does not escape the bound; it *relocates and minimizes* the memory: $\Theta(\log \delta)$ bits per dead-chain element inside live coordinates (plus the MRDT version store, which the model provides). The sharper, machine-witnessed form from our design sweep is the *conservation law* (§1): the sort key must retain dead ancestors' timestamps (the verdict information survives every encoding change: stamp, spine, tombstone, or denominator bits), and ties force separate *metadata* only under timestamp-oblivious minting; the timestamp-faithful mint carries the same bits inside the coordinate. Eight timestamp-oblivious mutation variants each have a named machine-checked countermodel (fold-verdict erasure; repair non-locality); the credential countermodel separates this design from all of them.

Verified datatypes. Isabelle RGA (Gomes et al. OOPSLA'17, op-based convergence), OpSets (list semantics specified by a total order of identified ops: arbitration-by-identity as specification), VeriFx (SMT, convergence properties), MET (TLA+, list CRDT bugs). In the MRDT line (Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25, this framework), no verified sequence with sequence-CRDT-grade ordering guarantees exists; our verified rehoming RGA (§1) is RA-linearizable against its own fold yet fails the sequential delete-order spec, since RA-linearizability certifies convergence to the datatype's *own* semantics and ordering intent must be specified and proved separately. The embedded-chain RGA delivers the missing artifact: RA-linearizability is a kernel-checked Lean theorem (§7); delete-order preservation, display stability and forward non-interleaving are proved at the model layer; and the RGA read-equivalence (the compaction theorem, §7) closes the intent question against the best-studied sequence semantics.

Treedoc's lesson, inverted. Treedoc re-balances geometry and must pay consensus for it; the principle that emerged from our ledger-based sibling design study (*reprojection without consensus is safe iff geometry has been demoted to a rendering*) is here taken to its fixed point: geometry is never reprojected at all. It is immutable, so it may safely be the only thing.

7 Mechanization status

The verification framework is the RA-linearizability metatheory for MRDTs (OOPSLA'25), mechanized in the Sal Lean development, the same framework in which twelve production MRDTs (the rehoming RGA among them) and a further half-dozen born-conditioned instances (the Peepul mergeable queue included) are verified end-to-end; file paths below are relative to that repository.

The capstone is proved (2026-07-15, kernel-clean on {propext, Classical.choice, Quot.sound}):

$$\texttt{embed\_ra\_linearizable3} \;:\; \mathrm{EReach}\ \Gamma\ C \to \mathrm{IsRALinearizable3}\ C$$

i.e. RA-linearizability per version at every honestly reachable configuration, *parametric in the code* Γ: the unary code, the binary delta code, and the flipped Elias-δ code are all proved `OrderedPrefixCode` instances, three encodings on one proof. The factoring is exactly this note's design–encoding split, which is itself evidence for the split:

1. **Model layer** (`Sal/MRDTs/RGA_Embed/`, six files, 0 sorry): the code-parametric kernel; commutations *to state equality* under `rc` = `Either` (`rc_non_comm'`); `ins_prefix_ghost`; coherence and chain-generation closures; display stability, with step stability (S2, delete-order preservation at its 'del' instance) proved *without any axioms* and merge pairwise stability (S4, the adopted contract); unique decodability (`coordOf_inj`); **the chain-lex theorem** (`display_iff_chainBefore` = Corollary 1); non-interleaving as subtree convexity (`subtree_convex`); Theorem 2(i) for all three codes (`binEnc_mono`/`binEnc_prefixFree`, `dEnc_mono`/`dEnc_prefixFree`, cross-validated against the Python codes by kernel `decide`); `native_decide` SPOTs where the rehoming RGA's delete-reorder witness gets the opposite verdict.

2. **Conditioned instance** (`Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/`, nine sections, 0 sorry, plus the Elias-δ inheritance corollary `EmbedRGA_EliasDelta.lean`), proved via the *mergeable-queue route*: a generic join lemma (`JoinLemma3At`: the merge exhibits its own linearization witness) plus an honest-reachability metatheorem (`HonestReach`), rather than the rehoming RGA's 55-file bespoke chain. The route applies exactly when states are canonical per event set, which is Theorem 1(i). Landed: the canonical sorted-list state; strictly-sorted extensionality (`esorted_ext`: sorted lists with the same members are equal, replacing any canonical-list formula); **fold-canonicity** (`e_fold_canon`: well-formed enumerations of one event set fold to the *same* state; Theorem 1(i) mechanized); the honesty core and its bridge to well-formedness; the merge membership characterization (`e_mergeL_mem`, OR-set survival with honesty closing the cross-branch-delete corner); the Join (`e_join_at`: the merge is its own linearization witness); the capstone; the honesty discharge from the framework's generation discipline (`eHonest_of_applicable`: applicable generation forces the honesty core, so per-op honesty needs no bespoke oracle); and the instance intent theorems: delete-order preservation *is* sublist stability (`eUpdate_del_sublist` is literally `List.filter_sublist`: a delete's successor state is a sublist of its predecessor, so the surviving display order is untouched; the merge preserves each input's survivor order on both sides).

The compaction theorem is proved (2026-07-15, kernel-clean): read ∘ embed = read ∘ $\mathrm{RGA}_\dagger$ on honest executions, as `rga_read_eq_embed_read` (`EmbedRGA_ReadEquiv.lean`): for every well-formed enumeration of an honest event set, *any* enumeration of the tombstoned fold's visible ids in the published RGA's own relational order (`visible_lt`) equals the embed fold's id sequence, with

element agreement (`embed_rec_readSeq`). The proof is *against the published definition*: the order core (`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`) shows `visible_lt` coincides with chain-lex on the shared birth tree (soundness by induction on derivations, completeness by realizing every chain prefix as an `afters`-reachable ancestor), and en route proves the published relational order total and asymmetric on the birth tree, facts the published side never had. This converts the design into a theorem *about* the published RGA: it admits a strictly-live compaction preserving every read. The lockstep rows of §4 are this theorem's tested form.

Provenance. `whiteboard/litmus/embed_tree.py` (the design, the timestamp-oblivious control, the credential countermodel, the IEEE 754 measurements; `python3 embed_tree.py` / `fp` / `code`); `litmus.py`, `pbt.py` (battery and DAG harness); `contest_tree.py` (lockstep harness, CE regressions); `whiteboard/retention-countermodel.pdf` (the retention arc that forced this design); `whiteboard/delta-tree-design.pdf` (the ledger-based sibling design and the invariants I1–I3); `whiteboard/litmus/related-work.md` (verified source list: Roh et al. JPDC'11; Attiya et al. PODC'16; Kleppmann et al. PaPoC'19; Weidner & Kleppmann TPDS'25 (Fugue); Gentle & Kleppmann EuroSys'25 (Eg-walker, arXiv:2409.14252); Preguiça et al. ICDCS'09 (Treedoc); Grishchenko & Patrakeev PaPoC'20 (Chronofold); Litt et al. CSCW'22 (Peritext); Gomes et al. OOPSLA'17; OpSets 2018; Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25; VeriFx ECOOP'23; Figma/Wallace fractional indexing; Weidner position-strings).